

DualPath: Accelerating Agentic LLM Inference by Harvesting Disaggregated KV-Cache Storage I/O

Yongtong Wu*
Peking University
DeepSeek-AI

Shaoyuan Chen*
Tsinghua University
DeepSeek-AI

Rilin Huang
Peking University

Yixuan Tan†
DeepSeek-AI

Yinmin Zhong*
Peking University
DeepSeek-AI

Mingxing Zhang
Tsinghua University

Xin Jin†
Peking University

Panpan Huang
DeepSeek-AI

Abstract

The performance of multi-turn, agentic LLM inference is increasingly dominated by KV-Cache storage I/O rather than computation. In prevalent disaggregated architectures, loading the massive KV-Cache from external storage creates a fundamental imbalance: storage NICs on prefill engines become bandwidth-saturated, while those on decoding engines remain idle. This asymmetry severely constrains overall system throughput.

We present DualPath, an inference system that breaks this bottleneck by introducing dual-path KV-Cache loading. Beyond the traditional storage-to-prefill path, DualPath enables a novel storage-to-decode path, in which the KV-Cache is loaded into decoding engines and then efficiently transferred to prefill engines via RDMA over the compute network. DualPath combines this optimized data path — which inherently avoids network congestion and avoids interference with latency-critical model execution communications — with a global scheduler that dynamically balances load across prefill and decode engines.

Our evaluation on three models with production agentic workloads demonstrates that DualPath improves offline inference throughput by up to 1.87× on our in-house inference system. It can also improve online serving throughput by an average factor of 1.96× without violating SLO.

CCS Concepts

• **Computer systems organization** → **Distributed architectures.**

Keywords

LLM Inference, KV-Cache, Disaggregated Storage, RDMA, Agentic Workloads

*Work done during internship at DeepSeek-AI.

†Corresponding authors.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
SIGCOMM '26, Denver, CO, USA

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2467-1/26/08
<https://doi.org/10.1145/3789240.3829159>

ACM Reference Format:

Yongtong Wu, Shaoyuan Chen, Rilin Huang, Yixuan Tan, Yinmin Zhong, Mingxing Zhang, Xin Jin, and Panpan Huang. 2026. DualPath: Accelerating Agentic LLM Inference by Harvesting Disaggregated KV-Cache Storage I/O. In *ACM SIGCOMM 2026 Conference (SIGCOMM '26)*, August 17–21, 2026, Denver, CO, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3789240.3829159>

1 Introduction

Large Language Models (LLMs) are rapidly evolving from single-turn chatbots [13, 32] and standalone reasoners [32] into *agentic systems* that can autonomously plan, invoke tools, and solve real-world tasks through *multi-turn interactions* [6, 20, 27, 41, 44]. In such settings, an LLM no longer serves isolated prompts; instead, it participates in long-running sessions where context accumulates over time [25]. As agentic applications become increasingly prevalent, multi-turn LLM inference has emerged as a critical workload in production systems, ranging from coding assistants [43, 48] to autonomous task agents [24, 55].

This paradigm shift in applications has driven a significant transformation in LLM inference workloads: from traditional human-LLM interaction to human-LLM-environment interaction, called the *agentic paradigm*. The typical pattern of human-model interaction involves users providing input, engaging in a few rounds of interaction with the LLM, and consuming the results generated by the LLM. By contrast, an agentic LLM may interact with an external environment, through tools such as a web browser and Python interpreter, over dozens or even hundreds of turns. Although each individual tool call or feedback is short (often hundreds of tokens), the context accumulates across turns and can grow to extreme lengths. As a result, agentic workloads become highly I/O-bound: the multi-turn, short-append pattern leads to very high KV-Cache hit rates — typically $\geq 95\%$ [5] — making the efficiency of KV-Cache loading, rather than pure computation, the dominant performance factor.

To improve throughput under agentic workloads, existing LLM inference systems have converged on a common set of architectural patterns: *layer-wise prefill* [14, 46], *prefill-decode (PD) disaggregation* [33, 51, 54], and *external KV-Cache storage* [15, 26, 34]. In these systems, prefill engines load the KV-Cache in a layer-wise

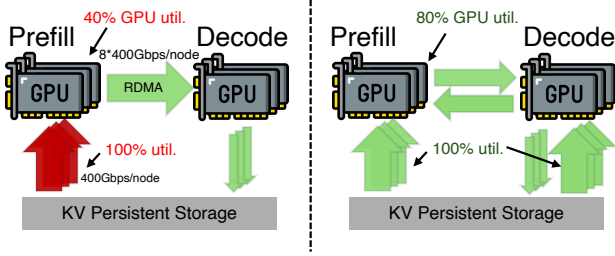


Figure 1: Existing bottleneck (left) and DualPath (right).

manner to accommodate as many requests as possible within a single batch. When prefill completes, decoding engines typically receive KV-Cache from prefill engines via a high-performance RDMA network. The decoding engines then generate tokens and store their KV-Cache in distributed storage to enable reuse across turns.

However, this architecture also introduces a critical limitation. As shown in Figure 1, prefill engines must load large volumes of KV-Cache from remote storage. As a result, *prefill-side storage network bandwidth* becomes the throughput bottleneck of the entire system, even though decoding engines often have substantial unused storage network bandwidth.

This imbalance reveals a fundamental inefficiency in existing designs: storage network bandwidth is unevenly utilized across engines. The bandwidth of prefill engines is persistently saturated, while decoding engines remain underutilized. Simply provisioning more bandwidth to prefill engines is costly and often impractical in general-purpose clusters. Therefore, it is promising to exploit and combine the available I/O bandwidth of all engines, rather than overloading prefill engines alone, to accelerate KV-Cache loading for agentic LLM workloads.

Prior studies have attempted to alleviate the KV-Cache loading bottleneck. Mooncake [34] caches KV-Cache in a distributed DRAM pool and employs an affinity-aware scheduler to maximize the DRAM KV-Cache hit rate. However, it cannot be used in memory-constrained scenarios, such as the rollout phase in RL, where DRAM is occupied to hold large training state that is offloaded from HBM. It is also not cost-effective in scenarios with enormous working sets (e.g., online serving), considering the cost comparison between DRAM and SSD. Other attempts reduce the amount of KV-Cache data to retrieve [16] and reduce the retrieval overhead [18, 47]. However, they do not solve the inherent inefficiency caused by storage I/O imbalance between different engines.

In this paper, we present DualPath, a new LLM inference system that rethinks KV-Cache loading in modern inference architectures for agentic workloads. The key insight behind DualPath is that KV-Cache loading does not have to be prefill-centric. While existing systems always load KV-Cache directly from storage into prefill engines, they cannot utilize the remote storage bandwidth of decoding engines. DualPath leverages this observation by enabling **dual-path KV-Cache loading**: in addition to the conventional storage-to-prefill path, KV-Cache can be loaded into decoding engines and then transferred to prefill engines via high-performance RDMA. By dynamically selecting between these paths, DualPath redistributes network load and alleviates prefill-side bandwidth pressure.

Realizing this design raises two challenges. First, introducing an extra loading path introduces complex traffic patterns and potential interference with collective primitives in model execution, which can degrade overall performance if unmanaged. Second, the system must decide online which loading path to use under dynamic and heterogeneous workloads, and ensure load balance across both GPUs and NICs simultaneously. To address these challenges, DualPath adopts (1) an optimized dual-path loading data path design, which introduces no inherent congestion under common P/D ratios, (2) a NIC-centric traffic management approach to isolate KV-Cache traffic from latency-sensitive model inference communications, and (3) a dynamic scheduling policy that jointly balances computation and network utilization across prefill and decoding engines.

We implement DualPath on top of a modern inference stack and evaluate it using representative agentic workloads with long contexts and high cache reuse. Experiments show that DualPath significantly improves system throughput and the first token latency, while maintaining the latency between tokens. In agentic inference scenarios, DualPath increases end-to-end throughput by up to 1.87× for offline inference, and improves the online serving throughput by 1.96× on average.

In summary, this paper makes three contributions:

- We identify the I/O-bound nature of multi-turn, agentic LLM workloads and show that KV-Cache loading dominates system performance under modern LLM inference architectures.
- We present DualPath, an inference system that introduces dual-path KV-Cache loading and leverages decoding-engine bandwidth to resolve prefill-side bottlenecks.
- We design and evaluate a workload-aware scheduling algorithm that dynamically balances computation and network resources, significantly improving balance on realistic workloads.

2 Background

2.1 LLM Inference Preliminary

LLM inference is becoming one of the most important system workloads recently. Popular LLMs utilize decoder-only transformer architecture, comprising stacked blocks with attention layers and feed-forward networks (FFNs). Attention layers enable token interactions within requests, while FFNs process tokens independently. The model predicts subsequent tokens based on preceding ones, storing attention keys and values as *KV-Cache* in HBM to avoid recomputations.

PD-disaggregated Inference. *Prefill-decode (PD) disaggregation* [33, 54] separates the prefill phase from the decode phase, assigning them to dedicated prefill engines (PEs) and decode engines (DEs), respectively. The two phases exhibit distinct compute and memory patterns: prefill is compute-intensive and batched, while decode is memory-bound and latency-sensitive. With PD disaggregation, PEs load the hit KV-Cache and perform prefilling; then, they transfer the KV-Cache to DEs, which perform autoregressive decoding. This design reduces interference between phases, enables stage-specific optimizations, and improves scalability, making it

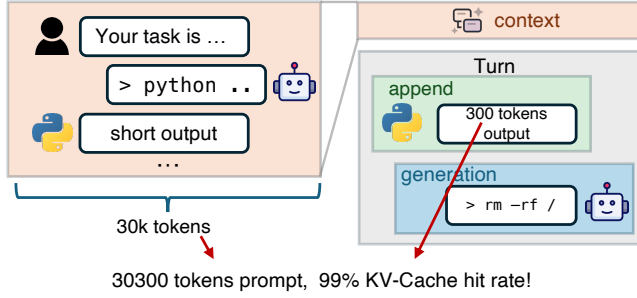


Figure 2: Agent trajectory example.

the de facto architecture for modern LLM serving. To support multi-turn conversations, the KV-Cache is often stored in distributed storage for reuse across turns.

Layerwise Prefill. Long-context prefill is bottlenecked by HBM capacity, as both activations and the KV-Cache for the entire batch must reside within it, forcing limited batch sizes and leading to poor GPU utilization. LayerKV [46] and PrefillOnly [14] address this problem by exploiting the strong locality in prefill computations: each layer requires only its own layer-specific KV-Cache. Consequently, the KV-Cache can be allocated and freed per layer, and the GPU holds only one layer’s KV-Cache for the forward batch. This increases the effective batch size (in tokens) by approximately a factor equal to the number of layers, boosting prefill throughput.

2.2 Agentic Use of LLMs

LLMs increasingly power *agentic* applications that perform multi-turn reasoning and interact with an environment (via e.g., terminal commands, code execution, or asking for human feedback) over long sessions. As shown in Figure 2, in a typical *turn*, the model receives a prompt composed by the previous *context* plus some newly *appended tokens* (often tool output or user input) and *generates* the next action or response. A single agent run is a *trajectory* of dozens or even hundreds of turns: the context grows turn-by-turn and can reach up to one million tokens [3, 8]. Because most of the context, typically >95% tokens in our traces, is reused across rounds, the vast majority of tokens in each round can hit the KV-Cache; only the newly appended context needs prefill computation. Due to the extreme length of agent trajectories, DRAM and HBM-based KV-Cache storage like Mooncake [34] can only store a small proportion of KV-Caches, necessitating the use of larger yet cheaper external SSD-based KV-Cache storage [10].

The agentic LLM inference workload is also prevalent in agent LLM training, which often adopts *reinforcement learning* (RL) approaches. In a typical RL training loop, the agent LLM first undergoes a *rollout* phase, where it is prompted to generate a large number of multi-step agent trajectories. These trajectories are then scored by a separate reward model. Finally, the LLM parameters are updated to increase the likelihood of high-scoring outputs and reduce the likelihood of low-scoring ones. During the rollout phase, substantial data (like reward model and optimizer states) is offloaded to host DRAM, further constraining the available DRAM for KV-Cache. This reinforces the need for external, high-capacity KV-Cache

storage that can accommodate long agentic rollout contexts efficiently.

2.3 Modern AI Data Center Architecture

Modern AI data centers are purpose-built logical supercomputers engineered to handle large-scale generative AI training and inference workloads. For example, in a standard NVIDIA DGX Super-POD [28], each node is equipped with 8 Hopper GPUs interconnected via high-speed NVLink. Each GPU is paired with a dedicated 400 Gbps compute NIC (CNIC, also known as east-west NIC), which maximizes inter-node communication bandwidth. Independent of the compute fabric, each node also features a storage NIC (SNIC, also known as south-north NIC) up to 400 Gbps, providing fast access to datasets, model checkpoints, and on-disk KV cache.

A fundamental principle of this architecture is that the compute network and the storage network are isolated from each other [51]. This separation is essential to maximize both storage and application performance. By isolating high-intensity east-west compute traffic between GPUs from storage traffic, the architecture prevents interference between them, and drastically reduces compute communication latency. This design also ensures that the inter-GPU communication remains highly reliable and predictable even when performing data-intensive tasks such as reading large datasets or writing multi-terabyte model checkpoints.

3 Bottleneck & Motivation

We observe severe GPU underutilization during agentic inference tasks. Our investigation reveals that KV-Cache loading speed is the bottleneck due to the limited bandwidth of the single storage NIC on each node. Analysis demonstrates that three decisive factors jointly cause this bottleneck, as discussed below.

First, agentic workloads exhibit high KV-Cache hit rates, which *require more I/O and less computation*, thus creating a severe I/O bottleneck. Agentic workloads are naturally long-context, short-append, and multi-turn. On each turn, the GPU needs to read the KV-Cache of the entire context from persistent storage and perform prefill computation for appended tokens. Our trace collected from representative coding tasks shows the mean number of rounds is 157, demonstrating the tendency of LLMs to engage in multi-turn interactions. The average context length is 32.7k, while the append length mean is only 429, which means a KV-Cache hit rate of 98.7%. In such a scenario, the cache-compute ratio, defined as the ratio of KV-Cache to load and the computation needed, is approximately 22 GB/PFLOP for DeepSeek-V3.2 [13], posing a significant bottleneck on storage bandwidth. Note that the KV-Cache size of DeepSeek MLA model is already highly optimized; for models with larger KV-Cache sizes (see Table 1), the situation is even worse. The ratio of DeepSeek-V3.2 is higher than DeepSeek-V3 [12], benefiting from its sparse attention design, lowering computation demands.

Second, the *hardware evolution trend* is not well suited for agentic inference workloads. In recent years, network bandwidth and HBM capacity have lagged behind the growth of GPU FLOPS, which drives us to run into memory and communication walls under agentic workloads. As shown in Figure 3, from NVIDIA Ampere to

Table 1: Cache-compute ratio with append length 429, across context lengths (16k–64k). KV-Cache data type defaults to FP8 unless specified.

Model	GB/PFLOP (16K–64K)
Qwen2.5-32B [38] (FP16)	117–267
GPT-OSS-120B [31]	47–95
Qwen3-235B-A22B [39]	39–60
DeepSeek-V3.2 660B [13]	13–36
DeepSeek-V3 660B [12]	4.8–5.8

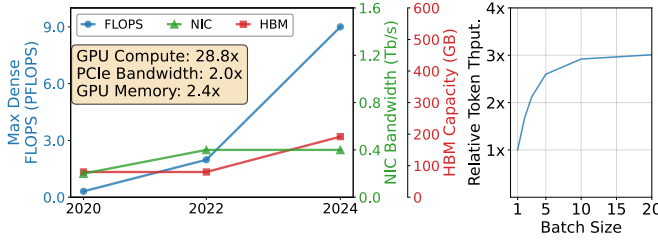


Figure 3: Left: Hardware trends of NVIDIA GPUs. Right: Relative token throughput with varying request batch size (each request has 30K context with 300 tokens appended).

Blackwell, the I/O-compute ratio decreases by 14.4x. Low NIC bandwidth limits KV-Cache loading speed, making GPUs idle. In addition, small HBM capacity limits the token batch size for GPU kernels [7, 11, 23, 50] to compute at the same time, hindering full utilization of compute units such as Tensor Core [14].

Third, existing LLM inference systems exhibit severe *storage network utilization imbalance* across different engine types. In prevalent PD-disaggregated systems, the KV-Cache for hit tokens is loaded exclusively by prefill engines directly from remote storage. This design centralizes all storage I/O pressure on the prefill-side SNICs, while the SNICs on decode engines remain largely idle. Consequently, the aggregate storage network bandwidth cannot be fully harnessed.

The above analysis demonstrates that the fundamental performance issue for agentic inference on PD-disaggregated architecture is the high I/O demand for KV-Cache retrieval and unbalanced storage network bandwidth utilization across inference engines. Meanwhile, we observe that the network traffic of the compute network, which has much larger aggregate bandwidth than the storage network, exhibits an intermittent pattern: collective operations used in model inference burst in sub-millisecond intervals. Therefore, an opportunity naturally emerges: we can utilize the SNIC bandwidth of decode nodes to load KV-Cache from storage, and transfer it back to the prefill nodes, utilizing the spare bandwidth of the faster compute network.

4 DualPath System Overview

To break the prefill-side storage I/O bottleneck, we propose a **dual-path loading** architecture that fundamentally rethinks how KV-Cache is retrieved in PD-disaggregated inference. Based on this architecture, we design and implement DualPath. DualPath adopts two widely-adopted techniques demonstrated in §2: (1) **PD Disaggregation** [33, 54], which separates prompt and decode processing

for better efficiency. (2) **Layerwise prefill**, which avoids HBM bottlenecks recognized by LayerKV [46] and PrefillOnly [14] on prefill engines and improves GPU utilization.

Our system consists of the following components:

- **Inference Engines.** Each engine manages one GPU. Engines are categorized into prefill engines (PEs) for prefill and decoding engines (DEs) for decode.
- **Traffic Manager (§5).** Each engine contains a traffic manager to conduct (1) Host-Device memory copies (H2D & D2H), (2) KV-Cache transfers between PEs and DEs, and (3) KV-Cache reads/writes from/to storage via the storage NIC. We adopt a CNIC-centric traffic management approach, detailed in §5, to prevent KV-Cache traffic from affecting communications in model inference.
- **Request Scheduler (§6).** A central scheduler that receives client requests and distributes them across engines. It is also responsible for dynamically distributing data traffic between two paths (Figure 4).

4.1 Dual-Path Loading

In addition to the conventional *storage-to-prefill* path, DualPath introduces a novel *storage-to-decode* path, allowing KV-Cache to be loaded first into a decode engine and then transferred to the prefill engine via high-bandwidth RDMA over the compute network. By dynamically distributing load across both paths, the system aggregates the storage NIC bandwidth of all engines — including otherwise-idle decode-side NICs — and eliminates the asymmetric bandwidth saturation that limits existing systems. This approach transforms the storage I/O from a single-bottleneck resource into a globally pooled and schedulable capacity. The exact data flows of the dual-path are described below.

To implement dual-path loading, DualPath allocates a small amount of DRAM as buffers on each PE and DE, called *PE buffer* and *DE Buffer*.

Prefill PE read path. First, the KV-Caches of hit tokens are read from persistent storage into the PE buffer (as Label 1 and 2 shown in Figure 4a). Before the computation of an attention layer, those KV-Caches of that layer are transferred to PE HBM (3 and 4) to compute the KV-Cache of cache-miss prompt tokens. Then, all KV-Caches of both hit and miss tokens are transferred to the DE buffer to form the complete prompt KV-Cache (5-7). This process (3-7) repeats n_{layer} times. During the prefill forward pass, transfers overlap with computation.

Prefill DE read path. The KV-Caches of hit tokens are first read into DE buffer (as Label 1 and 2 shown in Figure 4b). During PE prefill, KV-Cache for the corresponding layer is read from the DE buffer, also overlapping with computation (3-5). This process repeats n_{layer} times. After a layer’s computation completes, only the KV-Caches of miss tokens are transferred to DE buffer and merged with the existing hit token KV-Cache.

Decode Phase. After receiving the complete prompt KV-Cache in DE buffer (including loaded KV-Cache via PE read path and the KV-Cache of newly appended tokens), the decode phase begins. The DE first allocates HBM and performs host-to-device (H2D) transfers (Label 8 and 9 in Figure 4a; Label 6 and 7 in Figure 4b), then releases CPU memory before starting decode. The design of DE

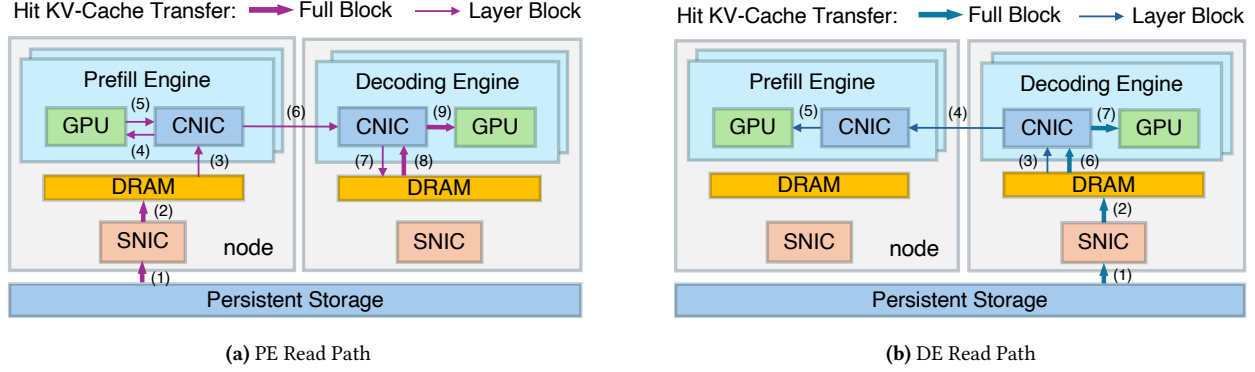


Figure 4: Dual path loading illustration. The scheduler dynamically distributes data traffic between the two paths.

buffer imposes bandwidth pressure on DRAM and CNIC (an extra H2D), which could be avoided by directly bypassing it via GPU Direct RDMA. However, since the generation length is typically short in this scenario, time-to-first-token (TTFT) accounts for a non-negligible portion of the total end-to-end request time. Introducing DE buffer helps reduce GPU memory usage. During decode, whenever a full block of tokens (e.g., 64 tokens) is accumulated, it is immediately persisted to disk.

Different Block Layouts. We adopt two different block layouts: *Full Block* and *Layer Block*, which contain all layers and a single layer, respectively. Detailed layout can be found in §A.4. For all interactions with storage, we adopt Full Blocks. In the PE read case, KV-Cache loading to PE HBM and transfer to DE Buffer occur in a layerwise streaming fashion, both using Layer Blocks. Similarly, for the DE read path, transfers from the DE Buffer to the PE HBM use Layer Blocks.

4.2 Bottleneck-Free Analysis

We demonstrate that the system can fully saturate all storage NICs without introducing compute-NIC or DRAM bottlenecks, under most reasonable P/D ratios. We assume a well-configured PCIe topology (each GPU–NIC pair shares a leaf PCIe switch), load-balanced task scheduling, no congestion on the computation network, and that storage read bandwidth is fully utilized.

Notation. Let P and D denote the number of prefill and decode nodes, respectively. Each node has g GPUs, each with one compute NIC of bandwidth B . The storage bandwidth per machine is $s \times B$ (shared by all engines on that machine); M is the memory bandwidth per machine.

Traffic per PE-DE pair. Under load balancing, storage NIC bandwidth is evenly shared. The traffic per pair for the PE read path (all steps in Figure 4a) is $T_p = Bs/(Dg^2)$; for the DE read path (Figure 4b) it is $T_c = Bs/(Pg^2)$. Link traffic is the sum over all pairs using that link.

PE CNIC Bandwidth Analysis. For PE CNIC, loopback traffic (i.e., H2D and D2H that does not traverse switches) exists, so the total traffic on the PCIe side is always greater than or equal to the switch-direction traffic, regardless of read or write operations.

Therefore, we only need to compute the pressure on the PCIe side. Read operations include PE paths (3) and (5), with total traffic over all pairs:

$$2 \times T_p \times Dg = 2Bs/g \leq B \quad (1)$$

Since $s \leq g$ always holds in practice, the read direction is always bottleneck-free. Write operations include PE path (4) and DE path (5), with total traffic:

$$(T_p + T_c) \times Dg = Bs/g \times (1 + D/P) \leq B \quad (2)$$

Then, we obtain:

$$P/D \geq \frac{s}{g-s} \quad (3)$$

DE CNIC Bandwidth Analysis. For DE CNIC, read operations include PE path 8 and DE paths 3/6, with traffic:

$$(T_p + T_c \times 2) \times Pg = s/g \times (P/D + 2) \times B \leq B \quad (4)$$

Then, we obtain:

$$P/D \leq \frac{g-2s}{s} \quad (5)$$

Write operations include PE paths 7/9 and DE path 7, with traffic:

$$(2T_p + T_c) \times Pg \leq B \quad (6)$$

This implies:

$$P/D \leq \frac{g-s}{2s} \quad (7)$$

DRAM Pressure Analysis. DRAM is half-duplex, so we sum the read and write pressures. For PE MEM, the pressure is $2sB$, which generally does not exceed memory bandwidth. For DE MEM, the pressure is $(3 + 2P/D)Bs$. Requiring the DE MEM pressure to be less than or equal to M , we obtain:

$$P/D \leq \frac{M/Bs - 3}{2} \quad (8)$$

Summary. Combining all the above analyses, we have:

$$\frac{s}{g-s} \leq P/D \leq \min \left\{ \frac{g-2s}{s}, \frac{g-s}{2s}, \frac{M/Bs-3}{2} \right\}. \quad (9)$$

For ($g = 8, s = 1$) with $M \approx 500$ GB/s and $Bs \approx 50$ GB/s, the bottleneck-free range is $\frac{1}{7} \leq P/D \leq \frac{7}{2}$, which covers most practical configurations.

4.3 Practical Challenges

The dual-path architecture fundamentally reorients data movement: KV-Cache can be loaded either directly from storage into prefill engines or indirectly via decode engines, thereby aggregating storage bandwidth across all engines and breaking the prefill-side I/O bottleneck. However, realizing this high-level design in a practical system introduces three interrelated challenges. We briefly outline these challenges below and refer the reader to the corresponding sections for details.

Fine-grained data transfer (§5). The layer-wise execution paradigm, while essential for overcoming HBM capacity limits, fragments the KV-Cache into numerous fine-grained blocks [33]. Transferring this multitude of fine-grained data chunks between storage, host DRAM, and GPU HBM must incur minimal overhead and seamlessly overlap with computation to realize throughput gains.

Traffic isolation (§5). The complex data path in DualPath introduces additional KV-Cache transfer traffic on both the compute network and PCIe links. A primary concern is that this traffic may interfere with existing latency-sensitive collective communication operations essential for model execution — such as AllToAll in expert parallel [52] and ReduceScatter/AllGather in tensor/context parallel. Since these collective communications are critical to end-to-end inference latency, a key challenge lies in exploiting spare I/O bandwidth without degrading model inference performance.

Dynamic load balancing (§6). As we are adopting two different paths for KV-cache loading, the system must promptly decide which path to use for each request. A naive policy could overload one path, recreating the original bottleneck. The traffic scheduler must balance multiple factors in real-time: storage NIC queue lengths, computational load on GPUs, and request workload characteristics.

5 CNIC-Centric Traffic Manager

Modern LLM inference systems employ a range of advanced data transfer technologies — such as on-chip CUDA copy engine and GPUDirect Storage [30] — to move data efficiently between storage, host memory, and GPU HBM. However, all these mechanisms can interfere with latency-sensitive collective communications (e.g., EP AllToAll) during model execution. This arises for two primary reasons: (1) such transfer technologies often operate over separate paths that do not share the same QoS controls as the compute network, and (2) existing GPUs do not support PCIe QoS [35], making it difficult to shield model inference communication from other traffic contending for PCIe bandwidth. Additionally, because the collective communications occur in rapid, sub-millisecond-level bursts, it is impractical to rely on a software-based traffic shaper to interleave lower-priority I/O operations between these high-priority traffic windows.

To address this, we propose a **CNIC-centric data transfer** approach: all data traffic in or out of a GPU, including local H2D/D2H copy, must go through the GPU’s paired CNIC with a GPUDirect

RDMA [29] data path. By consolidating all traffic onto the compute network, we can leverage the native QoS capabilities of compute network to enforce strict traffic differentiation. This approach is widely adopted in our production deployment, demonstrating its effectiveness.

5.1 Traffic Isolation with InfiniBand VL

In DualPath, we leverage InfiniBand *virtual lanes* (VLs) [4] to enforce isolation between different traffic classes. All model inference communication traffic is assigned to a dedicated high-priority VL, while all other traffic, including KV-Cache transfer, is mapped to a separate low-priority VL. We program the VL arbiters of all network switches and NICs with a weighted round-robin policy that reserves approximately 99.9% of total bandwidth to high-priority VL. A small fraction of bandwidth is allocated to low-priority VL to prevent starvation. This setup ensures that model execution remains virtually unaffected by KV-Cache traffic, while still allowing the KV-Cache data transfer to utilize the spare bandwidth of compute network.

Although our system is implemented on an InfiniBand fabric, the same principle can also be applied to Ethernet-based networks (e.g., RoCE) by leveraging IEEE 802.1Q Priority Code Point (PCP) and hardware packet queues.

5.2 CNIC-Assisted KV-Cache Copy

Existing GPU data transfer technologies include GPUDirect Storage [30], which loads KV-Cache from storage backend to GPU HBM, and CUDA copy engine, which directly copies host DRAM to GPU via PCIe. However, these methods fail to isolate KV-Cache traffic from high-priority latency-sensitive collective communications in model execution, severely degrading inference performance.

To solve the limitations of existing approaches, we adopt a CNIC-assisted H2D/D2H data path. For KV-Cache loading, we first read the KV-Cache into host DRAM from the storage backend. Then, we submit an RDMA Write request to the GPU’s paired CNIC to perform local H2D copy. Storing newly generated KV-Cache follows a symmetric process: it is first transferred to host DRAM via CNIC, then persisted to the storage backend over the storage network. This design establishes the CNIC as the central QoS scheduler for all GPU PCIe traffic, allowing its VL arbiter to prioritize the inference communication traffic and perform KV-Cache transfer using spare PCIe bandwidth.

Although this approach may appear to take a detour compared to GPUDirect Storage (which directly reads KV-Cache to GPU HBM) and CUDA copy engine (which directly copies host memory to GPU HBM), to our best knowledge, this is currently the only practical method to ensure that KV-Cache load/store does not degrade the performance of critical model-execution communication.

We also observe that CNIC-assisted H2D and D2H outperform the CUDA copy engine when handling a large number of small data chunks. Our measurements show that submitting a single copy operation via `cudaMemcpyAsync` incurs a latency overhead of approximately 5-7 μ s. We failed to further break down this overhead due to the closed-source nature of CUDA driver. In contrast, submitting one RDMA Write work request involves only a few mmio writes to NIC registers in user space and takes only around 1 μ s.

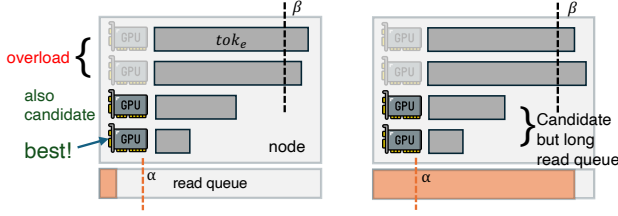


Figure 5: An illustration of Inter-Engine PE Scheduling. All eight GPUs are in the same PE engine group and the scheduler will choose the best.

Furthermore, the RDMA work submission overhead can be significantly amortized by leveraging *doorbell batching* [21].

6 Adaptive Request Scheduler

Although our theoretical analysis shows promising results, imbalanced load reduces hardware utilization, and in this scenario, we need to consider two dimensions of balance simultaneously: (1) NIC traffic, and (2) the utilization balance of GPUs. We divide scheduling into two levels: inter-engine scheduling, which assigns requests to a (PE, DE) pair and selects the read path (PE or DE) for each request; and intra-engine scheduling, which determines which requests are included in each forward batch for computation.

6.1 Inter-Engine Scheduling

We organize engines into groups to reduce the scheduler pressure. Only the engine rank 0, called *Leader Engine*, interacts with the scheduler. All engines of a group are all PEs or all DEs. All engines on one node are guaranteed to be in the same group. All engines in one group proactively fetch tasks together regularly. When fetching new requests, each engine e reports (1) seq_e , the number of requests assigned to it that have not yet completed; (2) the total token count tok_e over those seq_e requests; and (3) the disk reading queue length $read_qn(e)$ of the node $n(e)$ that engine e belongs to. GPU load, disk read load, and network load are all strongly correlated with token count. We therefore use token count as a proxy and aim to balance it across engines.

PE Scheduling. All requests arriving at the scheduler enter a waiting queue and are scheduled in a FIFO order. The scheduling algorithm is invoked when a PE group initiates a fetch request. An illustration of the inter-engine scheduling process is shown in Figure 5. We define two constants, short reading queue threshold α , and unfinished token upper limit β , measured in tokens. These are profiled in advance; adapting them online under variable storage performance is left as future work. All engines are split into three categories: (1) overloaded engines where $tok_e > \beta$; (2) engines on nodes with short disk reading queues where $read_qn(e) \leq \alpha$ and $tok_e \leq \beta$; and (3) engines on nodes with longer disk reading queues where $read_qn(e) > \alpha$ and $tok_e \leq \beta$. We do not assign new requests to overloaded engines. Second-category engines are prioritized over third-category engines because they reside on nodes with shorter disk reading queues, and lack of subsequent requests would easily lead to storage NIC underutilization.

Algorithm 1: Inter-PE Scheduling Algorithm

Data: Waiting queue Q , PE group G_{PE} , load metrics ($read_qn(e), tok_e$) reported by each engine e , where $n(e)$ denotes the node that engine e belongs to, constants α and β

Result: Assigned requests to PEs

Each engine e reports ($read_qn(e), tok_e$);

Classify all PEs into three categories;

$$C_1 \leftarrow \{e \in G_{PE} : tok_e > \beta\};$$

$$C_2 \leftarrow \{e \in G_{PE} : read_qn(e) \leq \alpha \wedge tok_e \leq \beta\};$$

$$C_3 \leftarrow \{e \in G_{PE} : read_qn(e) > \alpha \wedge tok_e \leq \beta\};$$

while Q is not empty **do**

$r \leftarrow$ head of Q ;

if $C_2 \neq \emptyset$ **then**

$pe^* \leftarrow \arg \min_{e \in C_2} tok_e$;

else

if $C_3 \neq \emptyset$ **then**

$pe^* \leftarrow \arg \min_{e \in C_3} tok_e$;

else

 Terminate this fetch request;

 Return assigned requests to Leader Engine;

break;

end

end

 Assign request r to PE pe^* ;

 Update $tok_{pe^*} \leftarrow tok_{pe^*} + \text{tokens}(r)$;

 Remove r from Q ;

end

We assign the current request to the PE with minimum tok_e in the second category if non-empty, otherwise in the third category if non-empty. After assignment, we update the selected PE's tok_e , then proceed to the next request in the waiting queue. If both categories are empty, we terminate this fetch request and return the already-assigned requests to the Leader Engine.

DE Scheduling Phase 1: across groups. DE scheduling is two-level and does not preserve global FIFO. There is a global waiting queue and a private queue per DE engine group. Incoming requests first enter the global queue. When a DE group fetches, *group-level* scheduling drains the global queue and assigns each request to the group whose total tok_e (sum over its engines) is minimum; this balances token count across groups and thus NIC and GPU load.

DE Scheduling Phase 2: within a group. Then we calculate the sum of remaining HBM for all DEs in the group and traverse from the head of the private queue to calculate how many requests can be scheduled assuming no HBM fragmentation. These requests form the set R . It is an upper bound that can be scheduled. Then, we calculate a high token threshold $Z = 1.05 \times (\sum_{r \in R} len_r + \sum_e tok_e)/e$.

Next, we try to pop the head of private queue and schedule it to a DE. Among DEs with sufficient remaining HBM for the request, we partition into (1) high-token DEs, where $tok_e + len(r) > Z$,

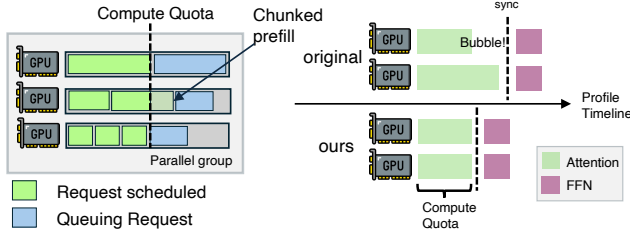


Figure 6: Intra-Engine Schedule. Left: compute-quota-based batch selection. Right: GPU timeline before and after applying compute quota.

and (2) the rest. We prefer category (2) to keep token count balanced; category (1) DEs already have higher GPU and NIC pressure. Within category (2) we choose the DE with minimum seq_e to balance request count; if category (2) is empty we choose the DE with minimum tok_e in category (1) to reduce HBM exhaustion and preemption risk. If no DE has sufficient HBM, the fetch ends and already-assigned requests are returned.

KV-Cache Read Task Scheduling. After selecting PE and DE for a request, we choose to read on the side with the shorter reading queue. It is probably better to split the request into two parts and read them from both sides, and we leave it as future work.

6.2 Intra-Engine Scheduling

Only PEs require intra-engine scheduling, as DEs always place all requests into the forward batch. An illustration of the intra-engine scheduling process is shown in Figure 6. Data parallelism is widely adopted for attention layers, especially for MLA models. Under such a parallel configuration, each GPU serves a different set of requests. It may lead to workload imbalance among all GPUs that must synchronize after the attention stage and enter the FFN stage together, causing GPU bubbles waiting for other peers. Therefore, we need to make sure they have similar attention layer execution times to minimize the waiting bubbles.

Layer Time Estimation. We use FIFO packing to decide how many requests to include in a forward batch. Each request in a forward batch is described by a pair ($cached, bsz$), where $cached$ is the number of tokens with KV-Cache already available (from storage hits or previous forward passes), and bsz is the number of tokens requiring KV-Cache computation in this forward batch. From these pairs, we compute the total theoretical computation for the attention layer and estimate its execution time. The relationship between theoretical computation and wall-clock time depends on hardware and parallel configuration, and can be fitted in advance through profiling as previous works [14] and [1].

Algorithm. We keep adding requests in FIFO order as long as the predicted attention layer execution time does not exceed a predefined upper bound, called the *compute quota*. If adding a request would exceed this bound, we perform binary search on bsz to find a smaller bsz' to fit in the remaining compute quota and perform chunked prefill for that request.

Table 2: Statistics of agent trace datasets.

MaxLen	Turns	Append	Gen	Total	Context
32K	60	608	148	28639	17183
48K	106	474	172	42607	25120
64K	157	429	176	55958	32721

7 Evaluation

7.1 Implementation

We implement DualPath based on our in-house inference framework. For CUDA kernels, our in-house framework adopts the combination of FlashMLA [23], DeepGEMM [11], and DeepEP [52], which aligns with the current mainstream open-source framework [22, 53]. The DualPath implementation involves approximately 5K lines of modifications on top of it. We adopt 3FS [10] as distributed storage and use an *io_uring*-like interface for kernel bypass.

7.2 Experimental Setup

Testbed. We conduct our experiments on a cluster of GPU servers with InfiniBand interconnection. Each server has 8 NVIDIA Hopper GPUs and dual processors. Additionally, each node is provisioned with eight 400Gbps RDMA NICs connected to InfiniBand network and one additional storage NIC connected to 3FS. The computation and storage networks are physically isolated. Our cluster-wide 3FS has no internal DRAM cache and can saturate the 400Gbps bandwidth of the storage NIC.

Models. We evaluate on three models: (1) DeepSeek V3.2 [13] 660B, an MoE model with DeepSeek Sparse Attention, denoted as *DS 660B*, (2) a 27B downscaled version of DS 660B, denoted as *DS 27B*, and (3) Qwen2.5-32B [38], a dense model with GQA, denoted as *Qwen 32B*. DS 660B and Qwen 32B correspond to the publicly released checkpoint on HuggingFace. DS 27B is our internal experimental model with a similar architecture to DS 660B. Detailed specifications are provided in §A.1.

Datasets. We collected three agent trace datasets from our production agentic RL training workloads with varying maximum context lengths (MaxLen). Each dataset contains 500 trajectories. The average interaction turns (Turns), average appended and generated tokens per turn (Append and Gen), total number of tokens (Total), and total number of context tokens (Context) are summarized in Table 2.

Baselines. We compare DualPath, denoted as Ours, against the following baselines:

- **SGL(MC):** SGLang [53] (commit 19089aa) with HiCache [36], Mooncake [34] Store enabled and 3FS as the storage backend, and Mooncake Transfer Engine for prefill-decode disaggregation. We did not run SGL(MC) for DS 27B because SGLang lacks support for this downscaled version.
- **Basic:** Our unmodified internal inference framework (detailed in §7.1). Comparing DualPath and SGL(MC) is unfair due to implementation differences. Therefore, we only report performance improvements from Basic to Ours.

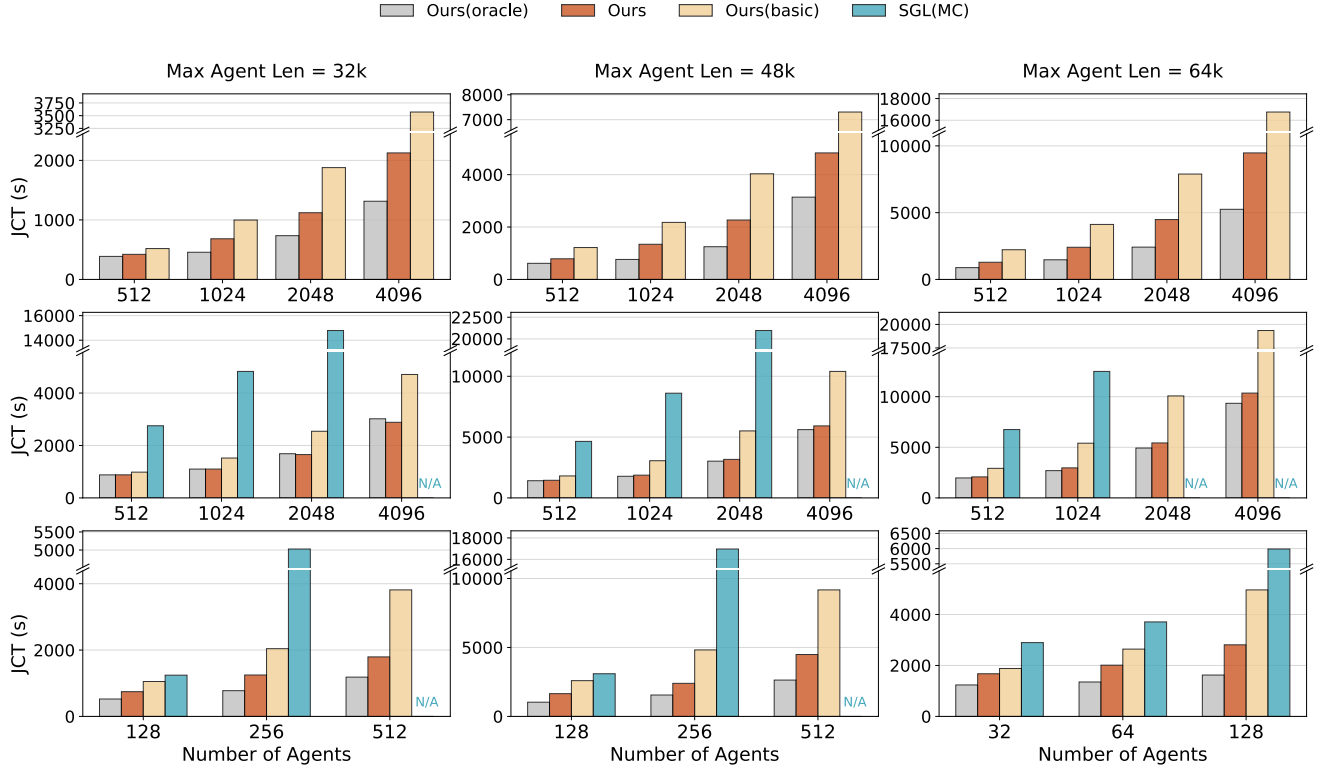


Figure 7: Offline inference performance under varying numbers of agents and maximum agent context lengths. Top: DS 27B. Middle: DS 660B. Bottom: Qwen 32B. N/A for running into an error before finishing.

- **Oracle:** Based on DualPath, we bypass all disk reads, D2H & H2D transfers, and inter-PD KV-Cache transfers. This configuration represents the theoretical performance upper bound assuming zero I/O overhead.

P/D Ratio and Parallelism. We default to 2P4D for DS 660B, 1P2D for Qwen 32B, and 1P1D for DS 27B (where 1P1D means one node for each side). For DS models, we use EP and DP. For Qwen 32B, we use DP only in DualPath, while SGL(MC) uses TP=8 since DP attention is not supported for this model in SGLang. Detailed configuration is provided in §A.3.

Metrics. For offline inference scenarios, we measure job completion time (JCT) for the entire task. For online serving scenarios, we measure TTFT, TTST (Time to the second token), and TPOT.

7.3 Offline Batch Inference

This section evaluates throughput performance in offline batch inference, which is the case of the rollout phase in RL training. In this scenario, n agents start to rollout simultaneously, and we measure the JCT when all requests have finished.

Varying Agents Batch Size & Max Agent Length (MAL). DualPath benefits more from larger batch sizes and longer MALs. Figure 7 reports JCT under different batch sizes and MALs. SGL(MC) encountered errors in our setup and failed to complete some large configurations (marked as N/A). On DS 660B, DualPath achieves

up to $1.87\times$ over Basic, and demonstrates performance with Oracle, indicating that KV-cache I/O is largely eliminated. On DS 27B, DualPath improves over Basic by up to $1.78\times$ but remains $1.09\text{--}1.85\times$ slower than Oracle due to limited storage bandwidth in 1P1D (Figure 8). For Qwen 32B, it shows similar trends as DS 27B.

Varying Append Length & Generation Length. DualPath has more advantages when append and generation tokens are short. Longer append lengths imply greater GPU compute pressure, and longer generation lengths lower KV-Cache loading pressure due to larger prefill gap time. To investigate the impact of this factor, we scale each round’s append length by a constant factor, and then truncate the whole trajectory at given MAL. The same holds for generation length. As shown in Figure 9, with append length increases, Basic performance gradually approaches DualPath and Oracle, while DualPath and Oracle performance changes only slightly, indicating that the bottleneck consistently lies in GPU compute pressure. Compared to Basic, DualPath achieves $1.82\text{--}1.99\times$ speedup at different append scales. The trend for generation length scaling is similar.

Varying Prefill-Decode Ratio. Across all ratios, DualPath demonstrates substantial performance gains compared to Basic. We conduct rollout experiments on DS 27B with 1P1D, 2P1D, and 1P2D prefill-decode ratios to characterize the impact of resource allocation between prefill and decode stages on overall system performance. As shown in Figure 8, DualPath achieves an average

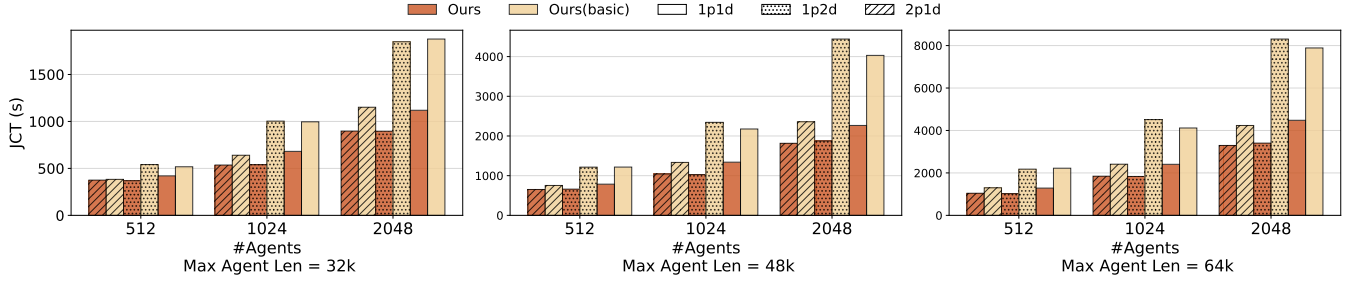


Figure 8: Impact of prefill-decode ratio on offline inference performance (DS 27B).

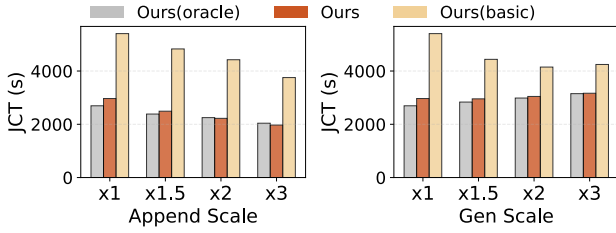


Figure 9: Left: varying append lengths (DS 660B, 64K context, 1024 agents). Right: varying generation lengths (DS 660B, 64K, 1024 agents)

speedup of 1.64 $\times$ across all configurations (up to 2.46 $\times$). Basic 1P1D and Basic 1P2D perform comparably; so do DualPath 1P1D and Basic 2P1D, as well as DualPath 2P1D and DualPath 1P2D. This occurs because each pair of systems has equivalent available storage bandwidth (Basic can only use prefill node storage bandwidth, while DualPath can utilize all nodes), which confirms that storage bandwidth is the dominant bottleneck in agentic scenarios.

7.4 Online Serving

Methodology. We evaluate system latency characteristics under varying agent arrival rates per second (APS). Agents arrive according to a Poisson process at a specified rate, with each agent commencing replay from round zero to its last round upon arrival. For our experiments, the SLO is set as $TTFT \leq 4$ seconds and $TPOT \leq 50$ ms. In the TPOT and TTST figures, data points exceeding the SLO threshold are omitted. Experiment termination is triggered when either: (1) TTFT exceeds 4 seconds, or (2) the system reaches steady state, defined as TTFT variation within a 150-second sliding window remaining below 5% compared to that 30 minutes prior. We do not conduct online experiments on Qwen 32B due to the experiment budget.

As shown in Figure 10, DualPath achieves higher APS capacity than Basic (1.67 $\times$ for DS 27B, 2.25 $\times$ for DS 660B). DualPath’s TTST is comparable to Basic, while TPOT shows that DualPath does not introduce additional decoding overhead compared to Basic. SGL(MC) exhibits anomalously low TTST, likely due to implementation issues where the first two tokens arrive at the client almost simultaneously. For DS 27B, all metrics exhibit trends similar to DS 660B. However, both Basic and DualPath show significantly

higher TPOT than Oracle, suggesting the overhead of basic P-D transferring is considerable in small model cases. We leave it as future work.

Average JCT for both models are presented in Figure 11. A detailed analysis of working set implications is discussed in §8. As shown in Figure 12 (left), DualPath maintains stable TTFT components across different APS, while Basic’s queuing time grows dramatically due to insufficient storage bandwidth. Note that the “Sch.” component includes queuing latency in the scheduler’s waiting queue, not just the scheduler’s computation overhead (which is under 10 ms).

7.5 Ablation Study

We conduct an ablation study to quantify the contribution of each technical component in DualPath. Experiments are performed under the offline inference setting with 64K MAL and agent batch size 1024 and 2048. The differences between Basic and Ours are grouped into three techniques: layerwise prefill, dual-path loading, and scheduling algorithm. We add the techniques gradually to demonstrate individual contribution. As shown in Figure 12, compared to Basic, adding layerwise prefill reduces JCT by 17.21% on average, alleviating PE HBM bottlenecks and hiding transfer overhead. Adding Dual Path Loading on top of layerwise prefill delivers the primary performance gains, reducing JCT by 38.19% on average compared to Basic, as it enables requests to read KV-Cache from either PE or DE, fully utilizing distributed storage bandwidth. Finally, employing our scheduling algorithm to decide KV-Cache loading paths achieves the best performance, reducing JCT by 45.62% compared to Basic, demonstrating the effectiveness of load-balanced scheduling across storage NICs.

Load Balance and Resource Utilization. DualPath’s scheduling algorithm improves load balance for both storage NICs and attention layer execution times. For storage NICs, our scheduling algorithm improves load balance from 1.53 to 1.18 compared to round robin scheduling (Figure 13). For attention layers, DualPath maintains the Max/Avg ratio as low as 1.06 during the first 5% of the task, reducing GPU idle bubbles (Figure 14). The statistics are collected from the +DPL and +Sched setting with 1024 agents in the ablation study (§7.5). The storage NIC metric is the ratio of maximum to average traffic across all storage NICs on three machines within a small time window, where 1.0 represents perfect balance. The attention layer metric is calculated among all GPUs in an expert parallel group for each forward. As the task progresses, both

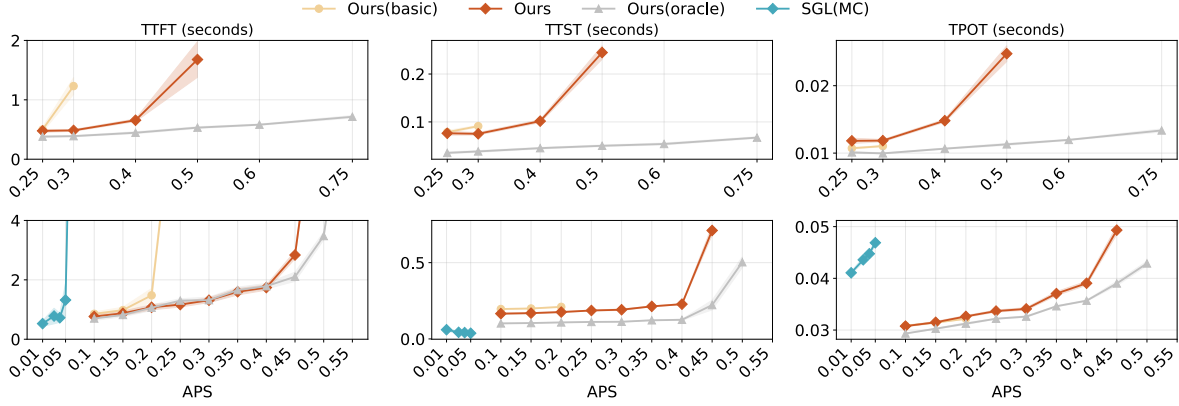


Figure 10: TTFT, TTST, and TPOT as functions of agent arrival rate (APS). Shadow means the fluctuation in the last 150s before experiments finish. Top: DS 27B, Bottom: DS 660B.

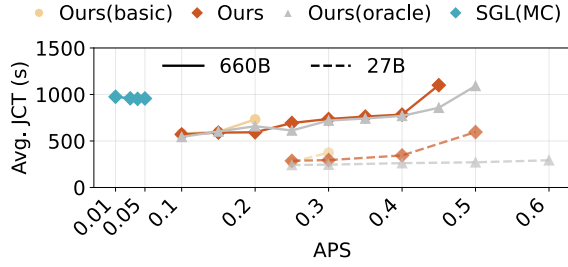


Figure 11: Average completion time of all trajectories versus arrival rate for online serving.

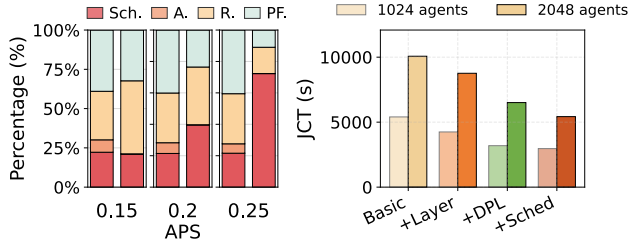


Figure 12: Left (§7.4): TTFT breakdown for online serving (DS 660B) across APSs, Sch. for scheduling, A. for allocating, R. for reading KV-cache, PF. for prefill. In each pair of pillars, the first is for DualPath and the second is for Basic. Right (§7.5): Offline inference ablation results (DS 660B, 64K context length). Layer, DPL, Sched stands for Layerwise prefill, Dual Path Loading, and scheduling, respectively.

ratios become meaningless due to underloaded system. Therefore, we do not show the tail phase of the workload. Load balance directly impacts peak resource utilization: the theoretical maximum NIC bandwidth utilization equals the reciprocal of the Max/Avg ratio. Our scheduling improves this from $1/1.53 = 65.4\%$ (round robin) to $1/1.18 = 84.7\%$, a 19.4 percentage-point improvement in achievable peak storage bandwidth utilization.

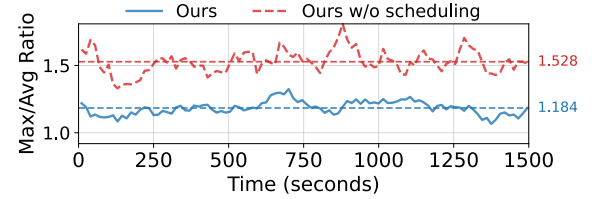


Figure 13: Load balance of storage NICs traffic.

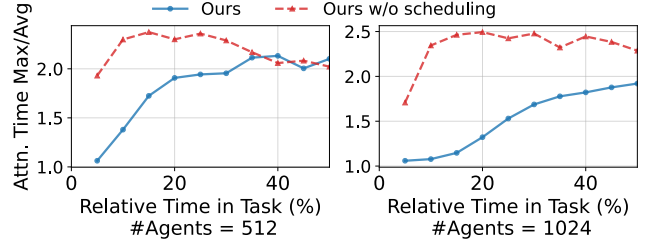


Figure 14: Load balance of attention execution time.

Table 3: Large-scale experiment results.

	Setting	JCT	TTFT	TTST	TPOT
Offline	2P4D, 2K agents	3,167s	–	–	–
	48P96D, 48K agents	3,201s	–	–	–
Online	2P4D, 0.4 APS	–	1.739s	0.228s	0.039s
	44P88D, 8.8 APS	–	1.847s	0.194s	0.036s

7.6 Large-Scale Scalability

We conduct both offline and online experiments using up to 1,152 GPUs to demonstrate production-level scalability (Table 3). For offline inference, scaling from 2P4D (2K agents) to 48P96D (48K agents) achieves near-linear speedup with comparable JCT (3,167s vs. 3,201s). For online serving, the 44P88D configuration achieves 22× throughput (8.8 vs. 0.4 APS) while maintaining similar latency. Across all

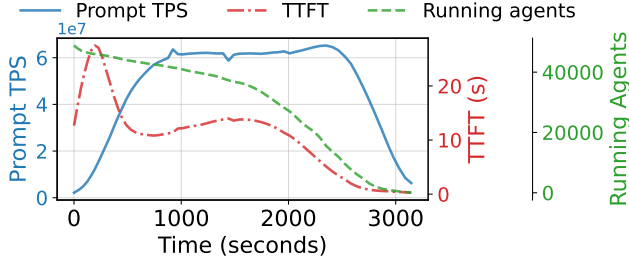


Figure 15: 48P96D offline inference metrics. $1e7$ is the scaling factor of Prompt TPS.

experiments, scheduler CPU usage remains below 10 cores, confirming it is not a bottleneck. Some detailed metrics over offline inference process are shown in Figure 15.

Due to the lack of fine-tuned parallelism settings and P/D ratios (which requires a substantial experimentation budget), the large-scale experiments do not demonstrate additional JCT or serving capacity gains compared to multiple small-scale units with equivalent cost. However, large-scale deployment remains important for the following reasons. First, it reduces fragmentation and provides greater flexibility for fine-tuning parallelism and P/D ratios. Second, large-scale deployment offers more scheduling opportunities to mitigate queuing latency under unpredictable bursty online requests. These observations suggest several directions for future work (§8.1).

8 Discussion

8.1 Potential Future Work

The workload of offline inference is highly dynamic. For example, in our agentic RL tasks, the workload depends heavily on the researchers' algorithm design, and the prefill stage typically experiences significantly higher pressure in the first half of execution than in the second half. Meanwhile, profiling these tentative experiments is costly, as some experiments are only run a limited number of times. Therefore, more adaptive and flexible approaches for parallelism and P/D ratio configuration are needed, such as simulators or online adjustment mechanisms. Second, the scheduling algorithm still has room for improvement, as we expect to achieve lower TTFT percentiles under large-scale deployment.

8.2 Working Set Analysis

Average JCT for both models are presented in Figure 11. Given an arrival rate λ (i.e., new trajectories per second) and mean JCT $\bar{T}$, the working set of the KV-Cache can thus be approximated as $\lambda \bar{T} \times total_len_{avg}/2$. In our setting of DS 660B serving, this value of DualPath ranges from 69 GB at APS 0.1 to 681 GB at APS 0.45.

In production, the working set would be larger since our evaluation assumes zero inter-arrival time and zero tool call latency. If JCT increases by r times due to these gaps, the system's APS capacity increases by r times (gaps do not stress LLM inference), causing the working set to expand by r^2 times. This would exceed available memory and reduce the distributed memory pool's hit rate. Such experiments require r times more machine hours and

r^2 times more storage (cost scaling as r^3), which we cannot afford given limited resources.

9 Related Work

Distributed Memory Cache Pools. Mooncake [34] builds a distributed DRAM pool for KV-Cache. TokenLake [42] introduces a unified segment-level prefix cache pool. Compared to them, DualPath targets storage backend directly, balancing the traffic among all SNICs, and reduces DRAM usage greatly without harming performance. DualPath can also be combined with a middle DRAM cache, but the performance gain is marginal.

KV-Cache I/O Optimization. Efficiently loading the massive KV-Cache from other caching tiers is a fundamental bottleneck in disaggregated LLM serving architectures. Prior work has approached this problem primarily from the perspective of a single data path. Strata [45] tackles I/O bottlenecks in hierarchical storage by co-designing GPU-assisted I/O with cache-aware scheduling. Other work, like KVPR [19] and TailorKV [49], mitigates bandwidth constraints (e.g., PCIe) on this path via recomputation overlapping and layer-granular hybrid quantization.

LLM Inference System. Recent years have seen many inference acceleration techniques, such as paged attention [22], chunked prefill, and hybrid batching [1, 17]. Prefill-decode disaggregated inference [33, 54] separates the prefill and decode stages onto different GPUs, reducing performance interference between them and allowing each stage to adopt distinct parallel strategies and hardware configurations, which unlocks substantial optimization opportunities. It has largely become the de facto standard for inference serving.

Attention Mechanisms. Attention mechanism allows tokens to interact with previous tokens in the sequence. There are many variants such as Multi-Head Attention (MHA) [40], Multi-Query Attention (MQA) [37] and Grouped-Query Attention (GQA) [2], Multi-head Latent Attention (MLA) [9]. For those attention mechanisms (denoted as *dense attention*), the ratio of computation and KV-Cache size for one token is a constant since both scale linearly with sequence length.

10 Conclusion

This paper presents DualPath, an agentic LLM inference framework that addresses the imbalance of KV-Cache reading under PD-disaggregated architecture through dual-path KV-Cache loading. By redistributing storage network load with workload-aware scheduling, DualPath achieves up to $1.87\times$ throughput improvement for offline inference. It also achieves $1.96\times$ higher agent runs per second on average in online serving.

This work does not raise any ethical issues.

Acknowledgments

We thank the anonymous reviewers and our shepherd for their valuable feedback. We would also like to thank our colleagues for their help: Wentao Zhang on scheduling topics, Liye Zhang, Shangyan Zhou, and Yuxuan Liu on networking, and Shunfeng Zhou on the file system. We use generative AI for grammar checks and language polishing.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 4895–4901.
- [3] Anthropic. 2026. Introducing Claude Opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>.
- [4] InfiniBand Trade Association. 2007. InfiniBand Architecture Specification Volume 1, Release 1.2.1.
- [5] Qiaoling Chen, Zhisheng Ye, Tian Tang, Peng Sun, Boyu Tian, Guoteng Wang, Shenggui Li, Yonggang Wen, Zhenhua Han, and Tianwei Zhang. 2026. CONCUR: High-Throughput Agentic Batch Inference of LLM via Congestion-Based Concurrency Control. arXiv:2601.22705 [cs.DC] <https://arxiv.org/abs/2601.22705>
- [6] Sadia Sultana Chowra, Riasad Alvi, Subhey Sadi Rahman, Md Abdur Rahman, Mohaimenul Azam Khan Raiaan, Md Rafiqul Islam, Mukhtar Hussain, and Sami Azam. 2026. From language to action: a review of large language models as autonomous agents and tool users. *Artificial Intelligence Review* (2026).
- [7] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations*. 35549–35562.
- [8] Google DeepMind. 2026. Gemini 3 Pro. <https://deepmind.google/models/gemini/pro/>.
- [9] DeepSeek-AI. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434
- [10] DeepSeek-AI. 2025. 3FS. <https://github.com/deepseek-ai/3FS>.
- [11] DeepSeek-AI. 2025. DeepGEMM. <https://github.com/deepseek-ai/DeepGEMM>.
- [12] DeepSeek-AI. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] <https://arxiv.org/abs/2412.19437>
- [13] DeepSeek-AI. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. arXiv:2512.02556
- [14] Kuntai Du, Bowen Wang, Chen Zhang, Yiming Cheng, Qing Lan, Hejian Sang, Yihua Cheng, Jiayi Yao, Xiaoxuan Liu, Yifan Qiao, Ion Stoica, and Junchen Jiang. 2025. PrefillOnly: An Inference Engine for Prefill-only Workloads in Large Language Model Applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*. 399–414.
- [15] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with Cached Attention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 111–126.
- [16] Shiwei Gao, Youmin Chen, and Jiwei Shu. 2025. Fast State Restoration in LLM Serving with HCache. In *Proceedings of the 20th European Conference on Computer Systems*. 128–143.
- [17] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. 2024. DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference. arXiv:2401.08671
- [18] Yifan Hu, Shi Qiu, Jianqin Yan, Hao Chen, Xintao Wang, Tang Lu, Guangtao Xue, and Yiming Zhang. 2025. TARDIS: A GPU-Centric KV Cache Service for Efficient LLM Inference. In *Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '25)*. Association for Computing Machinery, New York, NY, USA, 46–53. doi:10.1145/3725783.3764393
- [19] Chaoyi Jiang, Lei Gao, Hossein Entezari Zarch, and Murali Annamaram. 2025. KVPR: Efficient LLM Inference with I/O-Aware KV Cache Partial Recomputation. In *Findings of the Association for Computational Linguistics: ACL 2025*. 19474–19488.
- [20] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology* (2024).
- [21] Anuj Kalra, Michael Kaminsky, and David G Andersen. 2016. Design guidelines for high performance RDMA systems. In *2016 USENIX annual technical conference (USENIX ATC 16)*. 437–450.
- [22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [23] Jiashi Li and Shengyu Liu. 2025. FlashMLA: Efficient Multi-head Latent Attention Kernels. <https://github.com/deepseek-ai/FlashMLA>.
- [24] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Kong, Yile Wang, Hanfei Geng, Jian Luan, Xuefeng Li, Zilong Ye, Guanqing Xiong, Fan Zhang, Xiang Li, Mengwei Xu, Zhijun Li, Peng Li, Yang Liu, Ya-Qin Zhang, and Yunxin Liu. 2024. Personal LLM Agents: Insights and Survey about the Capability, Efficiency and Security. arXiv:2401.05459 [cs.HC] <https://arxiv.org/abs/2401.05459>
- [25] Weizhe Lin, Hui-Ling Zhen, Shuai Yang, Xian Wang, Renxi Liu, Hanting Chen, Wangze Zhang, Chuansai Zhou, Yiming Li, Chen Chen, Xing Li, Zhiyuan Yang, Xiaosong Li, Xianzhi Yu, Zhenhua Dong, Mingxuan Yuan, and Yunhe Wang. 2025. Towards Efficient Agents: A Co-Design of Inference Architecture and System. arXiv:2512.18337 [cs.CL] <https://arxiv.org/abs/2512.18337>
- [26] Yuhua Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. 2025. LMChe: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv:2510.09665
- [27] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. 2025. Evaluation and benchmarking of llm agents: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 6129–6139.
- [28] NVIDIA. 2023. SuperPOD: Next Generation Scalable Infrastructure for AI Leadership. <http://docs.nvidia.com/https://docs.nvidia.com/dgx-superpod-reference-architecture-dgx-h100.pdf>.
- [29] NVIDIA. 2026. Developing a Linux Kernel Module using GPUDirect RDMA. <https://docs.nvidia.com/cuda/gpudirect-rdma/index.html>.
- [30] NVIDIA. 2026. GPUDirect Storage Overview Guide. <https://docs.nvidia.com/gpudirect-storage/overview-guide/index.html>.
- [31] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925
- [32] OpenAI. 2025. Introducing GPT-5.2. <https://openai.com/index/introducing-gpt-5-2/>.
- [33] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Ñigo Gori, Saeed Maleki, and Ricardo Bianchini. 2025. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*. 118–132.
- [34] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies*. 155–170.
- [35] Andre Richter, Christian Herber, Thomas Wild, and Andreas Herkersdorf. 2016. Resolving Performance Interference in SR-IOV Setups with PCIe Quality-of-Service Extensions. In *2016 Euromicro Conference on Digital System Design (DSD)*. 454–462. doi:10.1109/DSD.2016.41
- [36] SGLang. 2026. SGLang HiCache. https://docs.sglang.io/advanced_features/hicache.html.
- [37] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150
- [38] Qwen Team. 2025. Qwen2.5 Technical Report. arXiv:2412.15115
- [39] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 6000–6010.
- [41] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
- [42] Bingyang Wu, Zili Zhang, Yinmin Zhong, Guanzhe Huang, Yibo Zhu, Xuanzhe Liu, and Xin Jin. 2025. TokenLake: A Unified Segment-level Prefix Cache Pool for Fine-grained Elastic Long-Context LLM Serving. arXiv:2508.17219 [cs.DC] <https://arxiv.org/abs/2508.17219>
- [43] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI] <https://arxiv.org/abs/2308.08155>
- [44] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences* 68, 2 (2025), 121101.
- [45] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Maitlody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. 2025. Strata: Hierarchical Context Caching for Long Context Language Model Serving. arXiv:2508.18572
- [46] Yi Xiong, Hao Wu, Changxu Shao, Ziqing Wang, Rui Zhang, Yuhong Guo, Junping Zhao, Ke Zhang, and Zhenxuan Pan. 2024. LayerKV: Optimizing Large Language Model Serving with Layer-wise KV Cache Management. arXiv:2410.00428
- [47] Jianqin Yan, Shi Qiu, Yina Lv, Yifan Hu, Hao Chen, Zhirong Shen, Xin Yao, Renhai Chen, Jiwei Shu, Gong Zhang, and Yiming Zhang. 2025. Phoenix: A Refactored I/O Stack for GPU Direct Storage without Phony Buffers. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*. 1267–1283.

- [48] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
- [49] Dingyu Yao, Bowen Shen, Zheng Lin, Wei Liu, Jian Luan, Bin Wang, and Weiping Wang. 2025. TailorKV: A Hybrid Framework for Long-Context Inference via Tailored KV Cache Optimization. In *Findings of the Association for Computational Linguistics: ACL 2025*. 20340–20359.
- [50] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasicki, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. arXiv:2501.01005
- [51] Chenggang Zhao, Shengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jiashi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shirong Ma, et al. 2025. Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1731–1745.
- [52] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>.
- [53] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*. Article 2000, 27 pages.
- [54] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
- [55] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854* (2023).

Appendices are supporting material that has not been peer-reviewed.

A Appendix

A.1 27B Model Specifications

In terms of overall model scale, the hidden dimension 2560, the intermediate size of dense layers is 12288, the number of hidden layers is 30, the number of attention heads is 32, the number of routed experts is 72, the MoE intermediate size is 1536, the number of activated experts per token is 6, the number of shared experts is 2, and the number of initial dense layer is 1. Regarding the index attention mechanism, the number of attention heads is 32, the head dimension is 64, the topk tokens for sparse attention is 1024. The LoRA compression for the Q matrix of both indexer and main attention is removed.

A.2 Agent Task Structure

To provide context for the dataset characteristics, we briefly describe the agent task structure, though this background is orthogonal to our system design. Each agent operates within a sandbox environment containing a code repository with known bugs and associated error messages. The agent is instructed via prompt to diagnose and fix the bug. The model possesses tool-use capabilities, invoking bash commands in the sandbox by emitting structured outputs. The agent and environment engage in multi-turn interactions, where each turn consists of a prompt (previous context concatenated with new information, most of which is tool output) and the model generating a subsequent tool invocation by decoding.

Each trajectory is a sequence of rounds; round i consists of appended tokens A_i and the number of generated tokens g_i . We use G_i to indicate the tokens generated in round i , which are not presented in our dataset. We define $Context_{i+1}$ as the concatenated list

of $A_1, G_1, A_2, G_2, \dots, A_i, G_i$. In round $i + 1$ of our replay, the agent concatenates the prompt as $Context_{i+1} + A_i$, and then sets proper sampling parameters to ensure it generates exactly g_{i+1} tokens, i.e., G_{i+1} . To generate additional agent trajectories, we sample an existing trajectory and prepend a synthetic round with random tokens as A_1 and $g_1 = 1$.

A.3 Experimental Configurations

Parallelism Details. For DS 660B, the prefill side adopts EP16, the decode side adopts EP32. For DS 27B, both sides adopt EP=DP=8. For Qwen 32B, we use DP=8 for both prefill and decode side when evaluating DualPath, Basic, and Oracle. For SGL(MC) with Qwen 32B, we use TP=8 since DP attention is not supported in SGLang.

Configuration Parameters. For DeepSeek models, DualPath allocates 80GB DRAM each node, and SGL(MC) use totally 1.5TB DRAM on every node. For Qwen 32B, due to the larger KV-Cache, DualPath allocates 320GB. Speculative decoding is disabled for all setting. We use 3FS as the storage backend for all configurations. The short reading queue threshold α (described in §6) is set to the number of tokens we can read during 3 seconds, and the unfinished token upper limit β is set to the number of tokens one GPU can process for 5 seconds. Those values are profiled in advance. Compute Quota Threshold is set to 300ms for all DualPath and Oracle baselines.

KV-Cache Hit Length Calculation. For all systems except SGL(MC), we limit the KV-Cache hits to only occur within a trajectory, and the hit length is calculated in the client because no eviction is needed. For SGL(MC), hit lengths are computed internally based on HiCache and Mooncake Store cache states.

A.4 KV-Cache Block Layout

Layerwise prefill reduces KV-Cache block size to $1/\text{layer}$ of the original, and makes the number of blocks larger to $\text{layer} \times$, posing challenges to transfer and storage performance. To overcome this, we design two distinct block types: *Layer Block* and *Full Block*. A Layer Block is a byte tensor with shape $[1, \text{tokens}, \text{bytes}]$ and stores one-layer KV-Cache for some tokens. The number of tokens is called *block_size*. *bytes* indicates the cache bytes needed per layer per token. Meanwhile, a Full Block has shape $[\text{layer}, \text{tokens}, \text{bytes}]$. This design enables us to avoid manual KV-Cache memory layout conversion throughout inference by simply concatenating n Layer Blocks to yield a Full Block. KV-Cache is stored in distributed storage using a trie structure, where each tree node corresponds to a Full Block.

A.5 Traffic Isolation Microbenchmark

We validate the effectiveness of VL isolation (§5) with a microbenchmark. Under EP16, we generate 20 GB/s of low-priority background traffic per port while measuring DeepEP [52] dispatch latency. Without background traffic, the latency is 131 μs ; with background traffic but no VL isolation, it degrades to 877 μs (6.7 $\times$); with VL isolation enabled, it remains at 141 μs . This confirms that VL-based priority scheduling effectively shields latency-critical model execution traffic from KV-Cache transfer traffic.